

04. 지원서 완전 이해 & 꼬리질문 방어 (★최우선)

면접 최대 리스크 = 내가 쓴 지원서를 내가 다 이해하지 못하는 것. 직무면접은 자소서/경력 기반 꼬리질문으로 진행된다(면접관 2:1). 아래는 내 모든 기술 주장을 밑바닥부터 다시 이해 → 30초 설명 → 꼬리질문 방어하도록 정리한 것.

사용법: 항목마다 ① 진짜 뭔가 ② 밑단 개념 ③ 30초 스크립트 ④ 꼬리질문 체인 ⑤ 메모리 브릿지 ⑥ 막히는 지점. 스크립트를 외우지 말고 ②를 이해하라 — 이해하면 어떤 꼬리질문도 답한다.

1. RDC Reset Dependency Analysis — false violation 87.7%↓ (47,159→5,795)

진짜 뭔가: RDC(Reset Domain Crossing)는 서로 다른 리셋 도메인에 속한 두 플립플롭 사이의 신호 전달 문제다. 송신 FF는 리셋 A로, 수신 FF는 리셋 B로 리셋된다고 하자. 리셋은 보통 비동기로 assert되므로, 리셋 A가 떨어지는 순간 송신 FF 출력이 클럭과 무관하게 변하고, 그 변화가 수신 FF의 클럭 엣지 근처에 도착하면 메타스터빌리티가 발생한다. (CDC가 '다른 클럭'이라면 RDC는 '다른 리셋'이 만드는 같은 종류의 위험.)

핵심 알고리즘(부분집합 판단): *aggressor*(송신)의 리셋 소스 집합 $\subseteq$ *victim*(수신)의 리셋 소스 집합이면 RDC 위반이 불가능하다. 이유: aggressor를 리셋시키는 모든 조건이 victim도 리셋시키므로, aggressor가 리셋될 때 victim도 항상 함께 리셋된다 → victim이 "동작 중에 변하는 값을 샘플링"하는 상황 자체가 없다 → safe → `rdc_false_path` 처리.

5단계 절차: ① reset source 식별(문서+RTL) → ② 의존성 시뮬레이션 → ③ 소스 간 의존 Matrix 구축 → ④ Matrix의 safe(0) 쌍 → RDC Tool용 TCL constraint 자동생성 → ⑤ 설계 변경 시 Matrix 자동 갱신.

왜 false alarm이 폭증했나: 사용 툴(Meridian RDC)이 기본적으로 모든 리셋 시나리오 조합을 가정 → 실제로는 동시에 일어나지 않는 리셋 조합의 위반까지 전부 보고 → 4.7만 건. 제약(어떤 리셋이 함께 동작하는지)을 주면 noise가 사라진다.

30초 스크립트: "RDC는 서로 다른 리셋 도메인 간 신호가 비동기 리셋 deassert 타이밍 때문에 메타가 나는 문제입니다. 툴이 모든 리셋 조합을 가정해 4.7만 건 false가 났는데, 저는 '송신 리셋 소스가 수신 리셋 소스의 부분집합이면 둘은 항상 같이 리셋되어 위반이 불가능하다'는 논리로 safe path를 자동 판정하는 5단계 매트릭스 방법론을 만들어 87.7% 줄였습니다."

꼬리질문 체인: - Q: 부분집합이면 정말 안전한가, 반례는? → A: 핵심 전제는 "aggressor 리셋 $\subseteq$ victim 리셋"이 모든 동작 모드에서 성립한다는 것. 그래서 ②의 시뮬레이션으로 리셋 간 실제 의존(어떤 리셋이 항상 함께 켜지는지)을 확인해 매트릭스를 만든다. 전제가 깨지는 모드가 있으면 그 쌍은 safe로 넣지 않는다. - Q: false negative(진짜 위반을 놓침)는 어떻게 막았나? → A: 가장 중요한 리스크다. 매트릭스를 보수적으로 — 시뮬레이션에서 독립적으로 deassert 가능한 리셋 쌍은 절대 safe로 두지 않고, 불확실하면 위반으로 남겨 사람이 본다. 즉 "확실히 안전한 것만 깎고 애매하면 남긴다"가 원칙. - Q: 왜 simulation으로 의존성을 봤나, formal은? → A: 리셋 소스의 논리적 의존(예: soft reset이 hard reset에 종속)을 빠르게 전수 확인하려 시뮬레이션을 썼다.

formal/assertion으로 매트릭스를 교차검증하면 더 견고하다(개선점). - Q: RDC와 CDC의 차이 한 줄? → A: CDC=다른 클럭, RDC=다른 리셋. 둘 다 비동기 변화가 캡처 엣지 근처에 와서 메타가 나는 문제.

메모리 브릿지: HBM Base Die-메모리 컨트롤러도 power-on reset/soft reset/JTAG reset 등이 얹혀 RDC noise가 크다. 같은 매트릭스 방법을 Peri/컨트롤러에 그대로 적용 가능.

⚠ **막히는 지점:** "recovery/removal timing"(리셋 deassert의 setup/hold)을 물으면 → 리셋도 클럭 엣지 대비 deassert 타이밍(recovery=setup, removal=hold 유사)이 지켜져야 하고, 안 지키면 메타. 이 용어를 `daily_study/E_sta` 에서 확인.

2. CDC SFR-to-HW — noise 95% 자동분류, review 5%

진짜 원가: SoC는 여러 클럭 도메인(GALS)이 공존한다. SW가 APB/AHB로 SFR(레지스터)에 값을 쓰는 **버스트랜잭션 자체**는 req/ack 핸드셰이크로 동기화된다. 그러나 그 SFR **출력이 logic 클럭 도메인으로 직접 fan-out되는 configuration path**(clock divider 값, mask, mode 등)는 핸드셰이크 보호 밖이다 → 각각이 CDC path. multi-bit이 동시에 바뀌면 **bit skew**로 의도치 않은 중간값이 인가될 수 있다.

동기화 기법(반드시 구분): - **quasi-static / false path:** 값이 logic disable 상태에서만 쓰이고 이후 불변 → 동기화 회로 불필요, 타이밍 false path. 단, SW 약속이 깨지면 버그 → assertion으로 static 검증 필요. (면적/전력 0이 장점) - **shadow register(double buffering):** bus 도메인과 logic 도메인에 이중 레지스터 → multi-bit **atomic** 업데이트. 면적 2배. - **MUX-enable(recirculation) 동기화:** 데이터는 직접 연결, **enable/load 신호만 2FF 동기화**. 데이터가 enable 도착 전 stable해야 함(open-loop). - **2FF synchronizer:** 1-bit 신호의 기본 동기화(메타 해소시간 확보). multi-bit엔 부적합(bit skew) → gray code/enable 방식.

내 접근: path마다 의도가 다른 게 본질 문제 → RTL flow에 **CSR spec 표준화 단계**(IP-XACT 기반)를 끼워 register별 의도를 spec으로 전달 → 거기서 CDC constraint/assertion 자동생성. 단, spec 채우는 부담을 줄이려 (1) legacy register 속성을 RTL+SW 사용패턴으로 **LLM 추론** (2) spec field 초안 보조 (3) CDC report를 register 단위로 **클러스터링**. AI는 sign-off 대체가 아니라 **설계자가 표준화 flow에 들어오게 돕는 보조**.

왜 ML 단독 분류는 실패했나(2년 전): CDC는 static 검증이 100% 정확해야 sign-off로 넘어가는데, ML 분류의 잔여 오류를 신뢰할 근거가 없었다. → 진짜 원인은 "분류 알고리즘"이 아니라 "**설계 의도가 EDA Tool까지 전달되지 않는다**"는 것. 그래서 spec-driven으로 의도를 전달하는 쪽으로 재정의.

30초 스크립트: "APB write 자체는 핸드셰이크로 동기화되지만, SFR 출력이 logic 도메인으로 가는 config path는 보호 밖이라 CDC가 됩니다. 대부분 quasi-static인데 일부 command성 register가 섞여 false가 폭증하죠. 저는 register 의도를 IP-XACT spec으로 표준화해 CDC constraint를 자동생성하고, spec 작성 부담은 LLM 추론으로 덜어 SFR-to-HW noise의 95%를 자동 분류했습니다."

꼬리질문 체인: - Q: quasi-static인지 어떻게 보장? SW가 약속 어기면? → A: 그래서 assertion으로 "logic enable 중엔 이 register 안 바뀐다"를 검증한다. 약속 위반 시 시뮬/형식검증에서 잡히게. - Q: multi-bit config는? → A: bit skew 위험 → enable 동기화 또는 shadow register로 atomic 보장. 무지성 2FF를 각 비트에 넣으면 오히려 오류. - Q: register naming(`_CFG` vs `_CMD`)으로 뭘 했나? → A: configuration(stable) vs command(reset/init/enable)를 naming으로 구분하면 모듈 단위 일괄 constraining이 가능 → report QoR 크게 개선. (보충노트의 핵심 아이디어) - Q: IP-XACT vs SystemRDL 왜? → 아래 11번 참조.

메모리 브릿지: HBM Base Die·DRAM Peri도 mode register(MRS)·config가 도메인을 건넌다 → 동일 spec-driven CDC.

3. RTL Sign-off Agent — false violation 20%↓, 사업부 1호

진짜 원가: Lint/CDC 등 RTL sign-off는 설계자에게 수백 rule·false 섞인 리포트 해석을 요구한다. 이를 **도메인 지식 기반 Agent**로 보조. 핵심 설계: rule 특성에 따라 분석을 분기 — ① 구조적으로 명확한 rule = **Tool DB 기반 deterministic logic** ② 설계 의도가 들어가는 rule = **RAG로 도메인 지식 주입한 LLM 또는 LLM 단독 추론**. 근거 출처별 **신뢰도 스코어링**으로 "왜 그렇게 판단했는가"를 사람이 검증 가능하게 남김. Tool↔Agent는 **MCP 기반 Skill**로 통일 인터페이스, Framework에 step-by-step planning + 결과검증.

밀단 개념: RAG(검색증강생성)=외부 지식을 검색해 LLM 프롬프트에 주입 → 환각 감소. MCP(Model Context Protocol)=LLM/Agent가 외부 Tool을 표준 인터페이스로 호출하는 규약. deterministic vs LLM 분기=확실한 건 규칙으로, 모호한 건 추론으로(정확도-유연성 트레이드오프).

30초 스크립트: "Lint/CDC sign-off의 false 부담을 줄이려 도메인 Agent를 만들었습니다. 구조적으로 판정 가능한 rule은 tool DB 기반 결정 로직으로, 설계 의도가 필요한 rule은 RAG-LLM으로 분기하고, 판단 근거에 신뢰도 스코어를 붙여 사람이 검증하게 했습니다. pilot에서 false 20% 줄였고 사업부 1호 사례입니다."

꼬리질문 체인: - Q: deterministic/RAG-LLM/LLM 단독을 가르는 기준? → A: rule이 tool DB의 구조정보(연결관계·속성)만으로 판정되면 deterministic, 설계 의도/명세 해석이 필요하다면 RAG(문서 있음)·LLM 단독(문서 없음). - Q: 잘못 분류한 case는? → A: 있을 수 있어 sign-off를 대체하지 않고 우선순위/출발점만 제시. 최종 판단은 사람. 신뢰도 낮은 건 반드시 review로. - Q: Synopsys LintAdvisor·Cadence ChipStack AI와 차이? → A: 벤더는 범용, 내 건 사업부 rule/DB/flow에 특화 + 신뢰도 스코어링으로 검증가능성 확보. (지금 벤더 솔루션 평가·도입도 참여)

메모리 브릿지: 메모리 설계 sign-off도 같은 false 부담 → AI Transformation 확산에 직접.

⚠ **막히는 지점:** "AI가 틀리면?" → 항상 "사람 판단 *아래* 보조도구, sign-off 대체 아님 + 신뢰도 스코어"로 답. (내 일관된 철학)

4. CPU IP Hardening — RTL Equivalence Check (Formality), Full/Quick 이중화

진짜 원가: Equivalence Check(LEC, 논리등가검증)은 두 설계가 기능적으로 같은지 형식적(수학적)으로 증명한다. 시뮬레이션이 아니다. compare point(레지스터·primary output·black-box 입력)를 매칭하고 각 point로 모이는 logic cone이 등가임을 증명. 합성 후(RTL vs gate), ECO 후, **restructuring 후(RTL vs RTL)**에 쏜다.

내 케이스: ARM 최신 코어를 삼성 공정에 맞게 SoC compiler(Defacto)로 **재구조화(restructuring)** → hierarchy가 자동 변형되며 parameter 전달 오류·define 소실·unrolling 등으로 원본과 달라질 위험 → **Formality로 변형 RTL ≡ 원본 증명**. 변형으로 인스턴스 경로가 바뀌므로 set_compare_rule 로 원본↔변형 경로를 매핑해야 하는데, SoC compiler의 mapping 파일을 파싱해 **compare rule을 자동생성**(human error

0화). - **Full EQ**: 전체 hierarchy·모든 compare point 정밀 검증 = sign-off용. - **Quick EQ**: 재구조화된 하위 계층을 **black-box** 처리하고 새 wrapper의 인터페이스 정합성만 빠르게 1차 확인 → 디버그 TAT 단축.

찾아낸 실제 이슈: ① parameter propagation — wrapper의 parameter type이 바뀌거나 하위로 전달 안 되고 fixed value로 변환되어 EQ fail → 벤더(Defacto) 개선요청+workaround ② 특정 SRAM instance에서 define 구문 소실 → 재현해 벤더 공식 bug fix 유도.

30초 스크립트: "ARM 코어를 SoC compiler로 재구조화하면 hierarchy가 변형되며 원본과 달라질 위험이 있어, Formality로 등가성을 sign-off했습니다. mapping 파일을 파싱해 compare rule을 자동생성했고, 전체 검증용 Full EQ와 변형 하위를 black-box로 빠르게 보는 Quick EQ를 이원화해 1,760만 line 설계를 입력 5→2 상황에서도 무결성 검증했습니다."

꼬리질문 체인: - Q: compare point가 뭔가? → A: 등가비교의 기준점 — 레지스터, primary output, black-box 입력. 두 설계에서 이 점들을 매칭하고 사이 logic을 등가 증명. - Q: Quick EQ에서 black-box하면 그 안은 검증 안 되는데? → A: 맞다. Quick은 wrapper 연결오류(port mismatch/direction) 조기검출용 1차. 최종 sign-off는 Full EQ로 그 안까지 본다. 속도-정확도 분리. - Q: 시뮬레이션과 뭐가 다른가? → A: 시뮬은 입력 벡터 일부만, EQ는 **모든 입력조합을 형식적으로 증명**. 등가성엔 EQ가 정답. - Q: non-equivalence 나오면? → A: failing point의 cone을 디버그 → 원인이 tool 변형 버그면 벤더, 의도된 차이면 compare rule/black-box 조정.

메모리 브릿지: 메모리 Digital IP도 합성·ECO·재사용 시 정합성 보장에 EQ 필수.

5. Backend EDA Tool Chain 자동화 (Altair FlowTracer) — PI/PD TAT 20%↓

진짜 원가: FlowTracer는 backend(PI=Physical Implementation, PD=Physical Design) **작업 의존성/잡 스케줄러**다(make의 EDA용 DAG runner). FDL(Flow Description Language)로 단계 간 의존성 그래프를 기술 → 독립 구간 **병렬화**, 단계 hand-off 시 **자동 trigger**로 idle 제거.

문제: 칩 복잡도↑로 backend tool 단일 runtime↑ → 전체 TAT 지속 증가. PI/PD 의존성 그래프를 그려 병렬 가능 구간 식별 + hand-off 자동화.

30초 스크립트: "칩이 커지며 backend 단계 runtime과 단계 간 대기시간이 TAT를 키웠습니다. FlowTracer로 PI/PD 의존성 그래프를 FDL로 기술해 병렬 가능 구간을 돌리고 hand-off를 자동 trigger해 idle을 없애 PI/PD TAT를 20% 줄였습니다."

꼬리질문 체인: - Q: 가장 까다로운 단계? → A: 의존성 정의 — 어떤 단계가 진짜 선행이고 어디서 병렬 가능한지 식별(잘못 병렬화하면 race/잘못된 입력). 실 디자인 PoC로 검증 후 도입. - Q: 왜 그 tool? → A: 사내 실 디자인 기반 PoC로 사용 시나리오·기존 flow 호환성 비교 후 결정.

메모리 브릿지: 메모리 PD/PI signoff도 긴 runtime → 동일 병렬화로 개발 TAT 단축.

6. SDC 검증·관리 신규 방법론 평가 (Gencellicon / Xcelium TCV)

진짜 원가: SDC = 타이밍 제약(클럭 정의, **MCP**=multicycle path, **false path**, I/O delay 등). SDC가 틀리면 (잘못된 MCP/FP) over-constrain(불필요 비용)하거나 real path를 놓쳐 **silicon 타이밍 버그**가 난다. → SDC 자체의 정합성 검증이 필요. - **Gencellicon(Siemens)**: MCP/FP assertion 생성 + SDC promotion → SDC가 맞는지 검증. - **Xcelium TCV(Timing Constraint Verification)**: formal로 타이밍 제약을 검증.

30초 스크립트: "MCP·false path 같은 SDC 제약 자체의 정합성 검증이 미비해 silicon까지 SDC 버그가 새는 사례가 있었습니다. Gencellicon으로 MCP/FP assertion을 생성·검증하고, formal 기반 Xcelium TCV도 평가해 실 타이밍 버그 이력이 있는 디자인으로 PoC했습니다."

꼬리질문 체인: - Q: MCP/false path가 뭔가? → A: MCP=출발-도착 간 여러 클럭 사이클이 허용되는 경로 (타이밍 완화), false path=실제로는 동작하지 않는 경로(타이밍 무시). 잘못 지정하면 버그. - Q: 두 솔루션 결정적 차이 한 줄? → A: Gencellicon은 assertion 생성/SDC promotion 중심, Xcelium TCV는 formal 제약검증 중심.

메모리 브릿지: 메모리도 고속 IF에 MCP/FP 多 → SDC sign-off 정합성이 수율·타이밍에 직결.

7. SoC 설계 통합 환경 (Perforce) — TAT 20%↓, Human Error 50%↓, 해외 3 site

진짜 원가: 설계물을 로컬 관리하다 버전 사고 발생 + IP→Top RTL의 일관된 Bottom-up flow·Constraint 관리 부재. → **Perforce 기반** workspace 자동생성·재동기화·Config 기반 RTL Release 모듈 개발, 파일리스트 작성 정책 정의, RTL symbol conflict 자동분석. 3개 해외 연구소에 전파·1년+ 트레이닝(3주 긴급 출장, Jira action item·PoC 운영).

30초 스크립트: "설계 버전사고와 IP→Top의 일관된 flow 부재를 Perforce 기반 통합환경으로 풀었습니다. workspace 자동생성·config 기반 RTL release·파일리스트 정책으로 TAT 20%, human error 50% 줄였고, 3개 해외 site에 방법론을 통일해 첫 양산 과제 kick-off를 일정 내 성공시켰습니다."

꼬리질문 체인: - Q: RTL symbol conflict가 뭔가? → A: 여러 IP를 합칠 때 module/define 이름 충돌 → 자동 분석·검출 도구로 통합 무결성 확보. - Q: 협업에서 가장 어려웠던 점? → A(자소서2와 동일 톤): 환경·PoC 불명확 → Jira action item + 명확한 RnR/PoC 지정으로 소통창구 마련.

메모리 브릿지: 메모리 R&D도 다거점(이천/청주/해외) → 설계 release·버전 무결성 동일 과제.

8. AI Skill 공유 플랫폼 — 500명+, 조직혁신상

진짜 원가: Python In-house tool 유지보수 effort↑를 LLM API·AI Coding Agent·사내 개발망으로 자체 **CI/CD + hazard 조기탐지**로 풀어 본인 속도↑ → 만든 Skill을 사업부에 공유하는 플랫폼화(Infra 협업·권한관리·결과 신뢰도) → 500명+ 사용.

30초 스크립트: "개인 효율로 만든 AI Skill이 동료에게도 유효하다고 보고, Infra와 협업해 수천 명 규모에서 안정 동작하는 공유 플랫폼으로 키웠습니다(현재 500명+). AI 도입이 조직 작업방식을 바꾸려면 인프라·권한·신뢰도 보장이 함께 가야 함을 배웠습니다."

꼬리질문 체인: - Q: 확산에서 가장 중요한 것? → A: 개인 효율≠조직 효율. Infra 협업·권한관리·결과 신뢰도 3가지. - Q: 보안/권한은? → A: 사내 개발망·권한관리 체계 안에서. (→ 05 보안 답변과 일관)

메모리 브릿지: SK하이닉스 메모리 R&D의 AI Transformation/CAD 확산에 같은 패턴 적용.

9. 학부 암호 모듈 HW 설계 (AES/ECC/DES) — AES 27k→10k GE (63%↓)

진짜 원가: Virtex-4 FPGA 검증 + 180nm Magnachip pre-verification. **PPA 최적화 2건:** ① Timing report로 critical path 상 불필요 MUX 제거 ② AES S-box를 256-byte LUT 대신 logic gate로 재구현해 메모리 비용 제거 → 면적 27,000→10,000 GE.

밀단 개념: AES S-box=바이트 치환표(256B). LUT(ROM/메모리)로 두면 메모리 면적, **composite field(GF((2⁴)²)) 산술 로직**으로 구현하면 면적↓(연산으로 대체). GE(Gate Equivalent)=NAND2 1개 면적 단위.

30초 스크립트: "AES를 FPGA·180nm로 구현하며 PPA를 뺏습니다. timing report로 critical path의 불필요 MUX를 제거하고, S-box를 256B LUT 대신 logic으로 재구현해 메모리를 없애 면적을 27k→10k GE로 63% 줄였습니다. 이때 PPA 최적화에 흥미를 느껴 현재 설계 방법론 일을 하게 됐습니다."

꼬리질문 체인: - Q: S-box를 logic으로? 원리? → A: S-box는 GF(2⁸) 역원+아핀변환. composite field로 분해하면 작은 연산 조합으로 표현 가능 → LUT보다 면적 효율(대신 지연↑ 트레이드오프). - Q: GE가 뭐가? → A: gate equivalent, NAND2 면적 기준 단위. - Q: FPGA와 ASIC(180nm) 차이? → A: FPGA=LUT 기반 재구성, ASIC=고정 게이트. pre-verification은 FPGA로 기능, 180nm 라이브러리로 면적/타이밍 평가.

메모리 브릿지: Peri 회로도 면적·타이밍 트레이드오프(LUT vs logic)가 동일하게 중요.

10. TFHE Key Switching IP — 1저자 논문, 1-cycle 단축

진짜 원가: FHE(완전동형암호)=암호화 상태로 연산(복호화 없이). TFHE 스킴에서 **Key Switching**=연산 후 ciphertext를 다른 키 기준으로 바꾸는 단계로, LWE dimension만큼 작은 연산이 반복되는 병목. 이를 HW 가속 대상으로 분리해 Verilog IP로 설계, **KeySwitch 본 연산을 LWE dimension 단위가 아닌 단일 cycle로 단축**.

밀단 개념: LWE(Learning With Errors)=격자 기반 암호의 기반 문제. dimension=벡터 차원(클수록 안전·느림). 병목 가속=병렬 datapath로 dimension 루프를 펼침.

30초 스크립트: "동형암호는 복호화 없이 암호상태로 연산하는데, TFHE의 key switching이 LWE dimension만큼 반복되는 병목입니다. 이걸 HW 가속 대상으로 떼어내 Verilog IP로 설계하고 본 연산을 단일 cycle로 단축해 1저자로 발표했습니다."

꼬리질문 체인: - Q: 1-cycle이 왜 의미 있고 한계는? → A: dimension 루프를 병렬화해 throughput↑. 한계는 면적·전력 증가(병렬도↔자원 트레이드오프), 메모리 대역폭. - Q: 왜 key switching이 병목? → A: 연산 후 noise/키 정렬을 위해 매번 수행, 작은 곱·합이 dimension만큼.

메모리 브릿지: datapath 병렬화·연산 가속 감각 → PIM/GDDR6-AiM·HBM 기반 AI 연산과 직접 연결(K_ai_pim 카드).

11. SFR 자동생성·검증 (Dream Fair 최우수상) — TAT 12일→수초, 47→3회 (93.6%↓), DVCon

진짜 원가: SoC flow에서 RTL↔SFR(문서) 불일치 시 여러 엔지니어 거치는 디버깅. → **IP-XACT 기반 스펙으로부터 SFR 문서+검증환경 자동생성**되도록 flow 자체를 변경. 핵심 IP 한 건으로 "spec→SFR 문서 수초 + RTL 정합성"을 회의 시연해 20여 명 동의 획득.

IP-XACT vs SystemRDL(반드시 구분): - **IP-XACT(IEEE 1685):** IP 패키징·레지스터맵·인터넥트를 기술하는 XML 메타데이터 표준(범용, 통합/툴 교환). - **SystemRDL: 레지스터 전용** 기술언어 → RTL·문서·C header·UVM 자동생성에 특화. - 내 선택(IP-XACT) 이유: IP 통합/툴 생태계 호환 + 레지스터맵 외 메타데이터까지 한 소스로. (SystemRDL은 레지스터만이면 더 직접적)

30초 스크립트: "RTL과 SFR 문서 불일치 디버깅이 큰 비용이라, IP-XACT 스펙 한 소스에서 SFR 문서와 검증환경이 자동생성되도록 flow를 바꿨습니다. 신입이 20여 명 얹힌 flow를 바꾸는 게 어려웠지만 핵심 IP 시연으로 설득해 TAT 12일→수초, error revision 47→3회로 줄이고 DVCon abstract·최우수상을 받았습니다."

꼬리질문 체인: - Q: IP-XACT vs SystemRDL 왜 IP-XACT? → 위 참조. - Q: 신입이 flow 바꿀 때 반발은? → A(자소서3 톤): 20여 팀 작업방식이 같이 바뀌어야 함 → 한 IP로 수초 정합성 시연 = "근거 있는 패기". - Q: 검증환경 자동생성은 뭘 만드나? → A: register field로부터 access test·기본값/속성 체크 등 UVM류 환경 골격.

메모리 브릿지: 메모리도 mode register/SFR 多 → spec-driven 자동생성으로 정합성·CDC까지(2번과 연결).

12. 공통 방어 원칙 (모든 기술 질문에 적용)

1. **모르면 모른다 + 추론:** "정확히는 모르지만 원리상 이럴 것 같습니다 — ..." (거짓 단정 금지, 면접관은 사고과정을 본다).
2. **수치는 항상 출처/기준과 함께:** "어떤 dataset/기준?" 꼬리질문 대비 (예: 95%는 SFR-to-HW path 한 SoC의 W_DATA 위반 기준).
3. **AI 철학 일관성:** 항상 "sign-off 대체 아님, 사람 판단 아래 보조 + 신뢰도".
4. **로직→메모리 브릿지로 착지:** 기술 설명 끝에 "메모리에선 ..."을 붙인다.
5. **약점 선제 인정:** "메모리 디바이스 디테일은 입사 후 빠르게 채우겠다, 다만 디지털 설계·검증·sign-off 문제는 메모리도 동일하다."

연계: 기술 기초는 `daily_study/D_cdc · E_sta · H_verif`, 민감 질문(왜 신입/삼성→하이닉스)은 `05_positioning.md`.